

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ ГОМЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ П. О. СУХОГО

Факультет автоматизированных и информационных систем

Кафедра «Информатика»

Специальность 1-40 04 01 «Информатика и технологии программирования»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту

на тему: **«Web-приложение онлайн-голосования в системе образования»**

Исполнитель: студент гр. ИП-41 Загорцев В. В.

Руководитель: Прокопенко Д. В.

Дата проверки: _____

Дата допуска к защите: _____

Дата защиты: _____

Оценка работы: _____

Подписи членов комиссии

По защите дипломной работы: _____

Гомель 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Анализ предметной области	6
1.2 Аналитический обзор существующих аналогов	7
1.3 Обзор технологий для реализации программного обеспечения	9
1.4 Анализ инструментальных средств автоматизации разработки и тестирования	11
1.5 Постановка задачи на дипломное проектирование.....	14
2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	16
2.1 Общая структура приложения.....	16
2.2 Функциональная модель <i>Web</i> -приложения	18
2.3 Информационная модель программного комплекса	22
ЗАКЛЮЧЕНИЕ	32
Список использованных источников	33

ВВЕДЕНИЕ

Современные образовательные учреждения активно внедряют информационные технологии для автоматизации различных процессов управления и взаимодействия между участниками образовательного процесса. Одним из таких процессов является проведение опросов среди студентов, преподавателей и администрации учреждения.

Традиционные методы проведения голосования, такие как бумажные анкеты или устные опросы, обладают рядом недостатков: значительными временными затратами, сложностью обработки результатов, возможностью ошибок при подсчёте голосов и недостаточной прозрачностью процесса. В связи с этим возникает необходимость создания автоматизированной системы онлайн-голосования, обеспечивающей удобство проведения голосований и достоверность результатов.

Одним из перспективных решений является использование технологии блокчейн. Она позволяет создавать системы голосования, в которых каждый голос фиксируется и не может быть изменён после записи, а также обеспечивается возможность независимой проверки корректности подсчёта голосов каждым участником.

Целью данной дипломной работы является разработка веб-приложения онлайн-голосования для системы образования на основе технологии блокчейн, обеспечивающего удобство использования, прозрачность и надёжность хранения данных.

В рамках дипломной работы будет разработана информационная модель платформы, включающая основные сущности системы: пользователей, голосования, варианты выбора. Также в системе будет реализована новостная лента для публикации актуальной информации для пользователей.

Результатом работы станет программный комплекс платформы онлайн-голосования, который может применяться в образовательных учреждениях для опросов студентов и преподавателей, оценки качества образовательного процесса, а также для организации голосований при проведении конференций, конкурсов, олимпиад и других образовательных мероприятий.

Задачами дипломной работы являются:

- изучение особенностей разработки клиент-серверных архитектур;
- проведение анализа технологии блокчейн и возможностей её применения в сфере онлайн-голосований;
- разработка архитектуры *Web*-приложения для проведения голосований с учётом различных образовательных сценариев;
- проектирование структуры базы данных для хранения необходимой информации;
- реализация программных модулей приложения, включая пользовательский интерфейс, авторизацию и аутентификацию пользователей, интеграцию со смарт-контрактами;
- проведение тестирования разработанного программного обеспечения.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Анализ предметной области

В образовательных учреждениях регулярно возникает необходимость принятия коллективных решений, связанных с организацией учебного процесса, деятельностью студенческого самоуправления и проведением различных мероприятий. Одним из основных механизмов принятия таких решений является голосование. Голосование позволяет учитывать мнение участников образовательного процесса, включая студентов, преподавателей и администрацию, и принимать решения на основе большинства голосов.

В системе образования голосования могут использоваться в различных ситуациях. Например, они применяются при выборе представителей студенческого самоуправления, проведении опросов среди студентов, выборе тем проектов, а также при проведении конкурсов, хакатонов и научных конференций. В таких случаях важно обеспечить удобство участия пользователей и корректность подсчёта результатов.

Традиционные способы проведения голосований в образовательных учреждениях, такие как бумажные анкеты или очные опросы, обладают рядом недостатков. К ним относятся значительные временные затраты на организацию и проведение голосования, сложность обработки результатов, возможность ошибок при подсчёте голосов, а также ограниченная доступность для участников, находящихся вне учебного учреждения.

С развитием информационных технологий широкое распространение получили системы электронного голосования. Электронное голосование представляет собой процесс подачи и подсчёта голосов с использованием информационных систем и сетевых технологий. Использование таких систем позволяет автоматизировать процедуру голосования, ускорить обработку результатов и обеспечить возможность участия пользователей независимо от их местоположения.

Однако большинство существующих систем электронного голосования основаны на централизованной архитектуре. Это означает, что все данные о голосовании, включая информацию о пользователях и их голосах, хранятся на серверах одной организации или оператора системы. Такой подход может вызывать вопросы, связанные с безопасностью хранения данных, прозрачностью процесса голосования и уровнем доверия со стороны участников.

Одной из основных проблем централизованных систем является возможность изменения или удаления данных при наличии доступа к серверной инфраструктуре. Пользователи не всегда имеют возможность убедиться в том, что их голос был корректно зафиксирован и учтён при подсчёте результатов. В условиях, когда важна прозрачность процесса принятия решений, такие ограничения могут снижать доверие к системе голосования.

В связи с перечисленными проблемами возрастает интерес к использованию децентрализованных технологий для реализации систем голосования. Одним из наиболее перспективных решений является применение технологии блокчейн. Блокчейн представляет собой распределённую систему хранения данных, в которой информация записывается в виде последовательности блоков и защищается с использованием

криптографических методов. Основным преимуществом данной технологии является невозможность изменения ранее записанных данных без согласия участников сети.

Использование блокчейн-технологии в системах онлайн-голосования позволяет повысить прозрачность и надёжность процесса голосования. Каждый голос может фиксироваться в системе в виде криптографической записи, что обеспечивает неизменяемость данных и позволяет участникам убедиться в корректности учёта их голосов. Кроме того, применение смарт-контрактов позволяет автоматизировать процесс регистрации голосов и подсчёта результатов.

Таким образом, анализ предметной области показывает, что существующие системы электронного голосования обладают рядом ограничений, связанных с централизованным хранением данных и недостаточной прозрачностью процедур. Использование современных веб-технологий в сочетании с блокчейн-технологией позволяет создать более надёжную и прозрачную систему онлайн-голосования, обеспечивающую удобство использования и возможность независимой проверки результатов.

1.2 Аналитический обзор существующих аналогов

С развитием цифровых технологий появилось большое количество систем электронного голосования, предназначенных для проведения опросов, выборов и принятия коллективных решений. Такие платформы позволяют автоматизировать процесс голосования, упростить участие пользователей и ускорить обработку результатов. Однако большинство существующих решений имеют централизованную архитектуру и обладают рядом ограничений, связанных с безопасностью, прозрачностью и возможностью независимой проверки результатов.

Одной из наиболее известных платформ онлайн-опросов является система *SurveyMonkey*. На рисунке 1.1 представлен интерфейс выбора цели использования данной системы.

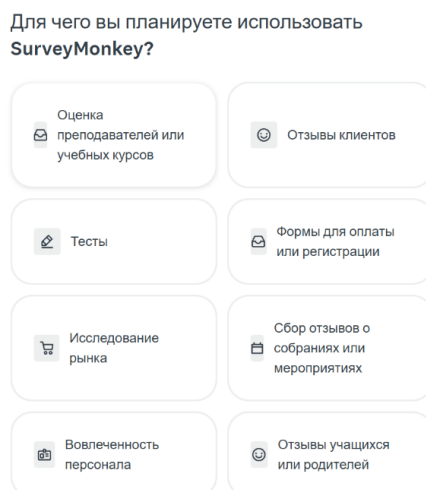


Рисунок 1.1 – Интерфейс выбора цели использования системы *SurveyMonkey*

После выбора цели пользователь получает доступ к основному функционалу системы, включающему инструменты для создания анкет, проведения опросов и анализа

полученных результатов. Рассмотрим процесс создания вопроса опроса, где пользователь может задать текст вопроса, выбрать тип ответа (например, множественный выбор), добавить варианты ответов для участников опроса, а также настраивать структуру и логику опроса. Данный интерфейс представлен на рисунке 1.2.

Рисунок 1.2 – Интерфейс создания вопроса в системе *SurveyMonkey*

Несмотря на удобство интерфейса и широкий набор функций, система *SurveyMonkey* является централизованной платформой. Все данные опросов и результаты голосования хранятся на серверах компании, а участники опроса не имеют возможности независимо проверить неизменность результатов голосования.

Другим распространённым инструментом является *Google Forms*. Данный сервис позволяет быстро создавать формы для опросов и голосований, распространять их среди участников и автоматически собирать результаты. Одним из основных преимуществ платформы является простота использования и интеграция с другими сервисами экосистемы *Google*. Однако, как и в предыдущем случае, система не обеспечивает полной прозрачности процесса голосования, поскольку результаты полностью контролируются владельцем формы.

Отдельную категорию составляют специализированные системы электронного голосования. Примером такой системы является *Helios Voting*. Данная платформа предназначена для проведения защищённых онлайн-выборов и использует криптографические методы для обеспечения анонимности и проверки голосов. Система предоставляет возможность независимой проверки результатов голосования, что повышает доверие участников. Тем не менее управление системой осуществляется централизованно, а хранение данных также контролируется операторами сервиса.

Проведённый анализ существующих систем показывает, что большинство современных платформ онлайн-голосования ориентированы на удобство использования и быстроту проведения опросов, однако не всегда обеспечивают достаточный уровень прозрачности и проверяемости результатов. Централизованное хранение данных

создаёт потенциальные риски изменения информации, а также снижает уровень доверия пользователей к системе.

В связи с этим возникает необходимость разработки систем голосования, основанных на децентрализованных технологиях. Применение таких технологий позволяет обеспечить неизменяемость записей о голосовании, повысить прозрачность процедуры и предоставить пользователям возможность независимой проверки результатов.

1.3 Обзор технологий для реализации программного обеспечения

Современные *Web*-приложения разрабатываются с использованием разнообразных технологий и инструментов, которые позволяют создавать масштабируемые, надёжные и безопасные информационные системы. Для реализации системы онлайн-голосований, обеспечивающей прозрачность и возможность независимой проверки результатов, был выбран комплекс технологических решений, включающий клиентскую часть, серверную часть, систему управления базами данных и блокчейн-технологии.

Архитектура системы построена по принципу клиент-серверной модели: пользователь взаимодействует с интерфейсом веб-приложения через браузер, при этом клиентская часть отвечает за отображение интерфейса и обработку действий пользователя, а серверная часть обеспечивает обработку запросов, хранение данных и реализацию бизнес-логики. Взаимодействие между компонентами осуществляется через сетевые протоколы передачи данных и специализированные *API*-интерфейсы.

JavaScript является одним из ключевых языков программирования для разработки *Web*-приложений. Он применяется для создания интерактивных интерфейсов, обработки событий и динамического обновления содержимого страниц. Основными особенностями языка являются динамическая типизация, автоматическое управление памятью и поддержка асинхронного программирования. С помощью технологии *AJAX JavaScript* обеспечивает обмен данными между клиентом и сервером без полной перезагрузки страницы, что повышает скорость работы интерфейса и удобство взаимодействия с пользователем.

Для построения пользовательского интерфейса используется библиотека *React*, которая широко применяется в современных *Web*-приложениях. *React* поддерживает компонентный подход: интерфейс делится на отдельные компоненты, каждый из которых реализует определённую часть функциональности системы. Такой подход облегчает разработку, повышает повторное использование кода и упрощает поддержку программного обеспечения. Механизм *Virtual DOM*, используемый в *React*, позволяет оптимизировать обновление интерфейса, изменяя только те элементы, которые действительно были изменены, что повышает производительность приложения.

Серверная часть разрабатывается на платформе *ASP.NET*, входящей в экосистему *Microsoft .NET*, и предназначенной для создания *Web*-приложений, *API*-сервисов и распределённых систем. Основным языком разработки является *C#*, современный объектно-ориентированный язык со строгой типизацией. *ASP.NET* предоставляет инструменты для обработки *HTTP*-запросов, управления пользовательскими данными, реализации механизмов безопасности и интеграции с базами данных.

Серверная часть реализует *REST API* (*Representational State Transfer Application Programming Interface*), обеспечивающее взаимодействие между клиентской и серверной частями приложения. В рамках *REST API* используются стандартные *HTTP*-методы: *GET* – получение данных; *POST* – создание новых записей; *PUT* – обновление существующих данных; *DELETE* – удаление данных. Передача данных между клиентом и сервером осуществляется в формате *JSON*, что обеспечивает гибкость архитектуры и возможность масштабирования системы.

Для хранения данных используется система управления базами данных *PostgreSQL*, которая обеспечивает надёжное хранение информации, поддержку транзакций и возможность выполнения сложных запросов. В базе данных хранятся сведения о пользователях, голосованиях, кандидатах или вариантах выбора, результатах голосований и другие необходимые данные системы.

Для взаимодействия серверной части приложения с базой данных используется *Entity Framework* – объектно-реляционный фреймворк (*ORM*), предназначенный для работы с реляционными базами данных в среде платформы *.NET*. *Entity Framework* позволяет разработчикам работать с базой данных на уровне объектов и классов языка программирования, что значительно упрощает процесс разработки и сопровождения программного обеспечения.

Одним из основных преимуществ использования *Entity Framework* является возможность применения подхода *Code First*, при котором структура базы данных формируется на основе моделей данных, описанных в программном коде. Это позволяет автоматически создавать и изменять структуру базы данных с помощью механизма миграций, что упрощает управление схемой базы данных в процессе разработки.

Кроме того, *Entity Framework* обеспечивает автоматическое формирование *SQL*-запросов, управление связями между сущностями, а также поддержку транзакций и механизмов отслеживания изменений данных. Благодаря этому разработчик может сосредоточиться на бизнес-логике приложения, не уделяя значительного внимания низкоуровневой работе с *SQL*-запросами.

Для обеспечения безопасного доступа к системе применяется механизм *JWT* (*JSON Web Token*). После успешной авторизации сервер формирует токен, который содержит зашифрованную информацию о пользователе и его правах доступа. *JWT* состоит из трёх частей: *Header* (заголовок), *Payload* (полезная нагрузка) и *Signature* (подпись).

- *header* содержит информацию о типе токена и используемом алгоритме подписи;

- *payload* включает данные о пользователе и его правах доступа, а также дополнительные сведения, например время жизни токена;

- *signature* формируется на основе заголовка и полезной нагрузки с использованием секретного ключа или пары ключей (в случае асимметричной криптографии), что обеспечивает целостность и защиту от подделки токена.

Токен сохраняется на стороне клиента и передаётся серверу при выполнении последующих запросов. Сервер проверяет подпись и расшифровывает *payload* для определения прав доступа пользователя, что позволяет безопасно управлять доступом к ресурсам системы без необходимости хранения сессий на сервере.

Для обеспечения прозрачности и проверяемости голосований используется технология блокчейн на базе сети *Ethereum*. *Ethereum* представляет собой децентрализованную платформу для создания и выполнения смарт-контрактов. Информация о голосах хранится в распределённом реестре блокчейн-сети, который состоит из последовательности блоков, связанных криптографически. После добавления данных в блокчейн изменить или удалить их невозможно, что обеспечивает высокий уровень безопасности и доверие пользователей к результатам голосований.

Solidity используется для разработки смарт-контрактов, которые автоматически выполняются при наступлении заданных условий. Смарт-контракты реализуют регистрацию участников голосования, фиксацию голосов, защиту от повторного голосования и автоматический подсчёт результатов. После публикации смарт-контракта его код становится неизменяемым, что гарантирует соблюдение правил работы системы.

Для взаимодействия пользователей с блокчейн-сетью используется криптокошелек *MetaMask*, позволяющий безопасно подписывать транзакции и фиксировать голоса в блокчейне. Клиентская часть приложения подключается к кошельку и может выполнять операции: подключение к кошельку, отправка транзакций голосования, получение информации о выполненных действиях и чтение данных из смарт-контрактов.

Использование библиотек *Web3.js* или *Ethers.js* обеспечивает удобную интеграцию *Web*-приложения с блокчейн-инфраструктурой и позволяет клиентской части напрямую взаимодействовать со смарт-контрактами.

1.4 Анализ инструментальных средств автоматизации разработки и тестирования

Для разработки программного комплекса системы онлайн-голосований будет использован комплекс современных инструментов и технологий, обеспечивающих удобство разработки, безопасное взаимодействие с блокчейном и управление данными. Применение специализированных средств повышает эффективность работы, облегчает управление проектом и обеспечивает более высокое качество программного продукта.

В качестве основной среды разработки выбран *JetBrains Rider*, а для работы с контейнеризацией и развёртыванием сервисов используется *Docker*. Хранение и работа с исходным кодом осуществляется через *Git* и платформу *GitHub*. Для управления реляционными базами данных применяется *DataGrip*, а тестирование *API* выполняется с помощью *Postman*. Взаимодействие с блокчейн-сетью реализовано через криптокошелек *MetaMask* и инструменты разработки смарт-контрактов *Hardhat* с сетью тестирования *Sepolia*. Для проектирования и визуализации архитектуры системы используется *StarUML*.

JetBrains Rider представляет собой современную интегрированную среду разработки (*IDE*) для создания приложений на платформе *.NET*, разработанную компанией *JetBrains*. Она предоставляет широкий набор инструментов для программирования на *C#*, включая разработку серверной части *Web*-приложений на *ASP.NET*, реализацию *REST API*, работу с базами данных и бизнес-логикой. Основные возможности *Rider* включают интеллектуальное автодополнение кода, встроенную отладку, анализ и поиск ошибок, интеграцию с *Git*, поддержку *Docker* и мощные инструменты рефакторинга.

Высокая производительность и удобный интерфейс позволяют эффективно работать с крупными проектами, а наличие множества плагинов расширяет функциональность *IDE*.

Docker является платформой для контейнеризации приложений, обеспечивающей запуск программного обеспечения в изолированных контейнерах, содержащих все необходимые зависимости, библиотеки и компоненты среды выполнения. Контейнеризация позволяет разработчикам упаковать приложение вместе с его окружением в единый переносимый образ, который может быть развернут на любой системе, поддерживающей *Docker*.

Использование *Docker* значительно упрощает процесс разработки, тестирования и развертывания программных систем. Благодаря контейнерам приложение работает одинаково независимо от операционной системы, конфигурации сервера или используемой инфраструктуры. Это позволяет устранить распространённую проблему несовместимости программного обеспечения между различными средами выполнения.

Одним из ключевых преимуществ *Docker* является изоляция приложений. Каждый контейнер работает в собственной изолированной среде и не влияет на работу других контейнеров или компонентов системы. Это повышает надежность и безопасность программного обеспечения, поскольку ошибки или сбои в одном контейнере не приводят к нарушению работы всей системы.

Важной частью экосистемы *Docker* являются *Docker*-образы и *Dockerfile*, которые позволяют описать процесс сборки контейнера и автоматизировать развертывание приложения. Это обеспечивает воспроизводимость среды выполнения и упрощает процесс переноса системы между различными серверами.

Для управления версиями исходного кода в процессе разработки используется распределённая система контроля версий *Git*. Данный инструмент позволяет отслеживать изменения в программном коде, фиксировать этапы разработки, а также при необходимости возвращаться к предыдущим версиям проекта. Применение *Git* обеспечивает удобную работу с ветками разработки, что позволяет параллельно реализовывать различные функциональные возможности системы и впоследствии объединять внесённые изменения.

Для удалённого хранения репозитория и организации совместной работы над проектом используется платформа *GitHub*, которая тесно интегрируется с системой контроля версий *Git*. Платформа предоставляет инструменты для размещения исходного кода в удалённом репозитории, управления ветками, отслеживания изменений и взаимодействия между разработчиками. Использование *GitHub* обеспечивает безопасное хранение программного кода, упрощает процесс коллективной разработки и способствует эффективному управлению жизненным циклом проекта.

DataGrip, разработанный компанией *JetBrains*, предоставляет графический интерфейс для работы с базами данных и выполнения *SQL*-запросов. Он поддерживает различные СУБД, включая *PostgreSQL*, применяемую в данном проекте. Основные возможности *DataGrip* включают выполнение запросов, визуальное управление таблицами, анализ структуры базы данных, автодополнение *SQL*-кода и редактирование данных, что облегчает разработку и администрирование базы данных приложения.

Postman позволяет тестировать *REST API*, отправляя *HTTP*-запросы к серверу и анализируя ответы. Он поддерживает различные методы запросов (*GET*, *POST*, *PUT*, *DELETE*), передачу данных в формате *JSON*, проверку авторизации и аутентификации, создание коллекций запросов и автоматизацию тестирования. Использование *Postman* ускоряет процесс разработки серверной части приложения.

MetaMask выступает криптовалютным кошельком для взаимодействия пользователей с блокчейн-сетью *Ethereum*. Он используется для подписания транзакций голосов в сети *Ethereum*, позволяя безопасно работать с блокчейн-инфраструктурой без хранения приватных ключей на сервере.

Hardhat – это инструмент для разработки, тестирования и развертывания смарт-контрактов в сети *Ethereum*. Он позволяет компилировать контракты на *Solidity*, запускать локальную тестовую сеть *Ethereum*, автоматизировать тестирование и деплой контрактов в тестовые и публичные сети. Одним из ключевых преимуществ *Hardhat* является возможность локального тестирования смарт-контрактов без совершения реальных транзакций, что существенно упрощает процесс разработки блокчейн-компонентов системы онлайн-голосований.

Alchemy (или *Infura*) – это облачный провайдер инфраструктуры для работы с блокчейн-сетями, такими как *Ethereum*. Данные сервисы предоставляют разработчикам доступ к удалённым узлам (нодам) блокчейна через *API*-интерфейсы, что позволяет взаимодействовать с сетью без необходимости самостоятельного развертывания и поддержки собственной ноды.

Использование подобных провайдеров значительно упрощает разработку децентрализованных приложений. Вместо запуска собственного узла *Ethereum* разработчик может отправлять запросы к сети через *HTTP* или *WebSocket API*, получая доступ к таким операциям, как отправка транзакций, чтение данных из смарт-контрактов, получение информации о блоках и событиях сети.

Одним из преимуществ таких сервисов является высокая доступность и стабильность инфраструктуры. Провайдеры обеспечивают постоянную синхронизацию узлов с блокчейном, распределение нагрузки и масштабируемость, что позволяет приложениям стабильно взаимодействовать с сетью даже при высокой активности пользователей.

StarUML – это приложение для создания *UML*-диаграмм, которое позволяет разработчикам создавать модели проектов, планировать архитектуру и дизайн приложений. *StarUML* имеет графический интерфейс пользователя, который предоставляет множество инструментов для создания и редактирования диаграмм, включая диаграммы классов, диаграммы последовательностей, диаграммы состояний, диаграммы активностей и многое другое. На рисунке 1.3 в виде выпадающего меню представлен весь список возможных диаграмм.

В *StarUML* можно создавать различные типы диаграмм, включая:

- диаграммы классов – используются для описания структуры классов и их отношений;
- диаграммы последовательностей – используются для описания взаимодействия между объектами и процессов, происходящих во времени;
- диаграммы состояний – используются для описания жизненного цикла объекта и его состояний;

- диаграммы компонентов – используются для описания структуры и отношений между компонентами системы;
- диаграммы развертывания – используются для описания физического размещения компонентов и системы в целом;
- диаграммы активностей – используются для описания последовательности действий и процессов в системе.

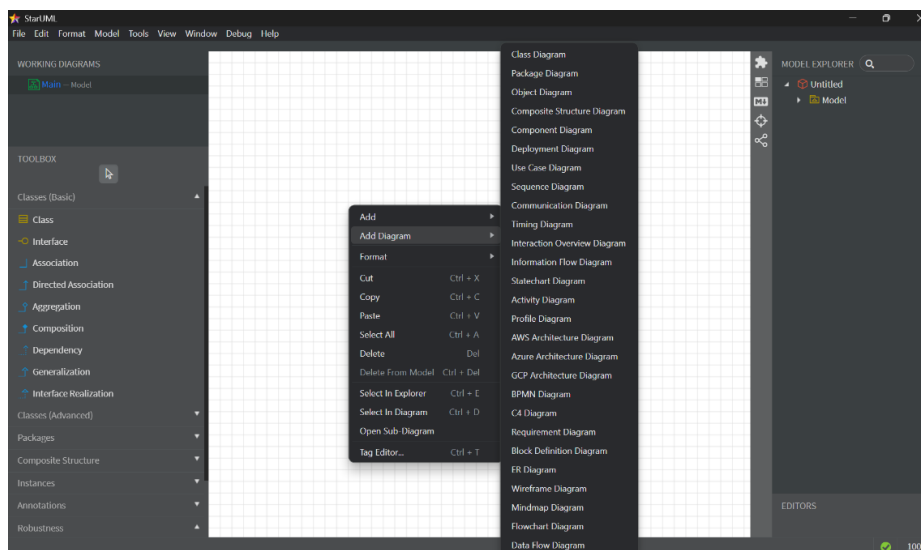


Рисунок 1.3 – Интерфейс главного окна *StarUml*

Данное приложение имеет множество функций, которые облегчают создание и редактирование диаграмм, такие как автоматическое выравнивание элементов, возможность группировки элементов, подсветка синтаксиса, подсказки и многое другое. Кроме того, *StarUML* позволяет пользователю создавать свои собственные элементы, шаблоны и плагины, чтобы улучшить функциональность и адаптировать инструмент под свои потребности.

1.5 Постановка задачи на дипломное проектирование

Цель разработки: разработать веб-приложение, предназначенное для автоматизации процесса проведения онлайн-голосований в системе образования.

Данный программный комплекс предназначен для использования в образовательных учреждениях для организации голосований, опросов и принятия коллективных решений среди студентов, преподавателей и администрации. Особенностью разрабатываемой системы является использование технологии блокчейн, обеспечивающей прозрачность, неизменяемость и надежность хранения данных о голосовании.

Для администраторов система предусматривает следующие функции:

- управление пользователями системы;
- назначение ролей пользователям;
- создание и управление голосованиями;
- модерация контента и голосований;

- просмотр статистики и результатов голосований;
- управление новостями системы.

Для преподавателей голосований приложение предусматривает следующие функции:

- создание голосований и опросов;
- настройка параметров голосования (дата начала и окончания, варианты ответов, тип голосования);
- редактирование и управление созданными голосованиями;
- просмотр результатов голосований;
- публикация информационных материалов и новостей.

Для студентов приложение предусматривает следующие функции:

- просмотр списка доступных голосований;
- ознакомление с описанием голосований;
- участие в голосовании с выбором одного или нескольких вариантов ответа;
- просмотр результатов завершённых голосований;
- получение уведомлений о новых голосованиях и публикации результатов.

Приложение должно иметь следующую структуру и функциональность:

- работать как веб-приложение, основанное на архитектуре REST;
- обеспечивать разграничение прав доступа пользователей в зависимости от их роли;
- предоставлять удобный пользовательский интерфейс для создания и проведения голосований;
- обеспечивать автоматический подсчёт голосов и формирование статистики;
- обеспечивать защиту от повторного голосования и несанкционированного доступа;
- предусматривать использование технологии блокчейн для фиксации результатов голосования, что позволяет обеспечить неизменяемость данных и возможность проверки подлинности голосов;
- быть кроссплатформенным и доступным через современные веб-браузеры;
- обеспечивать хранение и обработку данных пользователей и результатов голосований.

В результате разработки должно быть создано веб-приложение, позволяющее организовывать и проводить голосования в образовательной среде, обеспечивая удобство использования, прозрачность процесса голосования и автоматизированную обработку результатов.

2 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Общая структура приложения

Архитектура программного обеспечения разрабатываемой системы онлайн-голосования будет построена по принципу многоуровневой клиент-серверной архитектуры. Такой подход позволит разделить пользовательский интерфейс, бизнес-логику приложения и уровень хранения данных, что повысит масштабируемость, безопасность и удобство сопровождения системы. На рисунке 2.1 будет представлена архитектура взаимодействия компонентов системы онлайн-голосования.

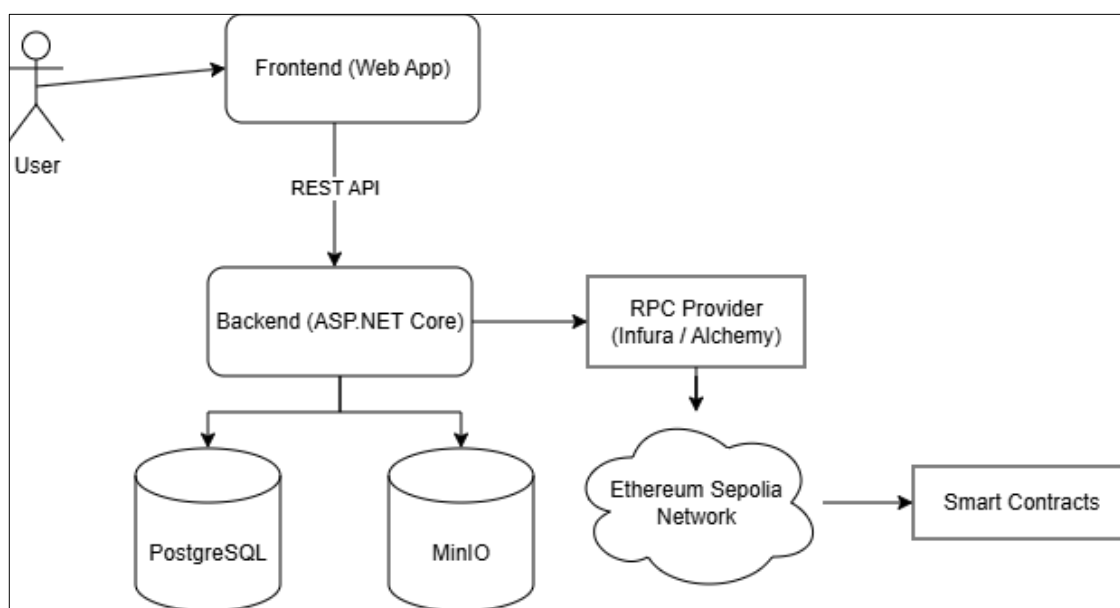


Рисунок 2.1 – Архитектура веб-приложения онлайн-голосования

Основными компонентами архитектуры системы являются клиентская часть (*frontend*), серверная часть (*backend*), база данных, система хранения файлов, а также блокчейн-инфраструктура, обеспечивающая прозрачность и неизменяемость результатов голосования.

Клиентская часть приложения реализована в виде веб-приложения на основе библиотеки *React*. *React* используется для построения интерактивного пользовательского интерфейса и организации взаимодействия пользователя с системой. Интерфейс приложения состоит из набора компонентов, каждый из которых отвечает за отображение отдельного элемента интерфейса, например списка голосований, формы участия в голосовании или новостной ленты.

Компоненты *React* взаимодействуют друг с другом с использованием механизма свойств (*props*) и состояния (*state*), что позволяет управлять отображением данных и обновлять пользовательский интерфейс при изменении состояния приложения. Клиентская часть взаимодействует с серверной частью системы посредством *REST API*, отправляя *HTTP*-запросы для получения и передачи данных.

Серверная часть системы будет реализована на платформе *ASP.NET Core* и будет выполнять функции обработки запросов от клиента, реализации бизнес-логики приложения и взаимодействия с внешними сервисами. Сервер обрабатывает операции регистрации и авторизации пользователей, управление голосованиями, обработку результатов голосования, а также управление новостями системы.

Кроме того, серверная часть отвечает за взаимодействие с блокчейн-сетью *Ethereum*. Для этого используется специальный *RPC*-провайдер (например, *Infura* или *Alchemy*), через который сервер отправляет запросы к сети и выполняет операции взаимодействия со смарт-контрактами.

В разрабатываемой системе будет использоваться комбинированный подход к хранению данных, при котором часть информации будет храниться непосредственно в блокчейн-сети, а часть – вне блокчейна (*off-chain*) в реляционной базе данных *PostgreSQL*. Использование блокчейна целесообразно для хранения критически важных данных, связанных с процессом голосования, таких как записи о поданных голосах, идентификаторы голосований и результаты подсчёта голосов. Размещение этих данных в блокчейне позволит обеспечить их неизменяемость, прозрачность и возможность последующей проверки достоверности результатов голосования.

В то же время значительная часть данных системы будет храниться вне блокчейна в традиционной базе данных. К таким данным относятся сведения о пользователях системы, параметры голосований, текстовые описания, новостные публикации, а также вспомогательная информация, необходимая для корректной работы приложения. Хранение подобных данных вне блокчейна позволит повысить производительность системы, снизить стоимость выполнения операций в сети блокчейн и упростить обработку больших объёмов информации.

Для реализации хранения данных вне блокчейна в системе будет использоваться система управления базами данных *PostgreSQL*. В базе данных будут храниться сведения о пользователях, голосованиях, вариантах ответов, результатах голосований, а также данные новостной ленты. Использование реляционной базы данных позволяет обеспечить надёжное хранение информации, поддержку транзакций и возможность выполнения сложных запросов к данным.

Для хранения файлов будет использоваться объектное хранилище *MinIO*. Оно будет применяться для размещения различных файловых данных системы, таких как изображения, прикрепленные материалы новостей или другие документы. *MinIO* обеспечит удобный доступ к файлам через *API* и позволит эффективно управлять хранением файловых данных.

Отдельным компонентом архитектуры является взаимодействие с блокчейн-сетью *Ethereum Sepolia*. Данная сеть используется для размещения смарт-контрактов системы голосования. Смарт-контракты обеспечивают регистрацию голосов пользователей, предотвращение повторного голосования и автоматический подсчёт результатов.

Для взаимодействия с блокчейном серверная часть приложения использует *RPC*-провайдер, через который выполняются запросы к сети *Ethereum*. Это позволяет отправлять транзакции, получать информацию о блоках и событиях сети, а также взаимодействовать с развернутыми смарт-контрактами.

2.2 Функциональная модель *Web*-приложения

Для разработки качественного программного приложения необходимо чётко определить роли пользователей системы и доступный им функционал. Для этого требуется определить, какие объекты входят в предметную область проектируемой системы, а также какие логические связи между ними существуют. Для формирования такого представления используются логические модели предметной области. Целью построения логической модели является получение наглядного графического представления логической структуры исследуемой системы. Для стабильной работы программного обеспечения необходимо чёткое распределение ролей пользователей, на основе которых формируются функции взаимодействия с системой.

Актёр – это множество логически связанных ролей в *UML*, выполняемых при взаимодействии с системой или её отдельными компонентами. Актёром может выступать человек или внешняя система, которые взаимодействуют с разрабатываемым приложением.

Прецеденты представляют действия, выполняемые системой в интересах актёров. Проще говоря, прецедент – это описание последовательности действий, которые выполняются системой и приводят к получению для актёра определённого результата. Один актёр может использовать несколько прецедентов, и один прецедент может быть связан с несколькими актёрами. Совокупность всех прецедентов определяет функциональные возможности системы.

В разрабатываемой системе онлайн-голосования выделяются три основные роли пользователей: пользователь, организаторы администратор. Каждая из этих ролей обладает определёнными функциями и правами доступа в системе.

В качестве среды для разработки и отображения функциональной структуры программы будет использована *StarUml*. На рисунке 2.3 представлена *Use Case* диаграмма приложения.

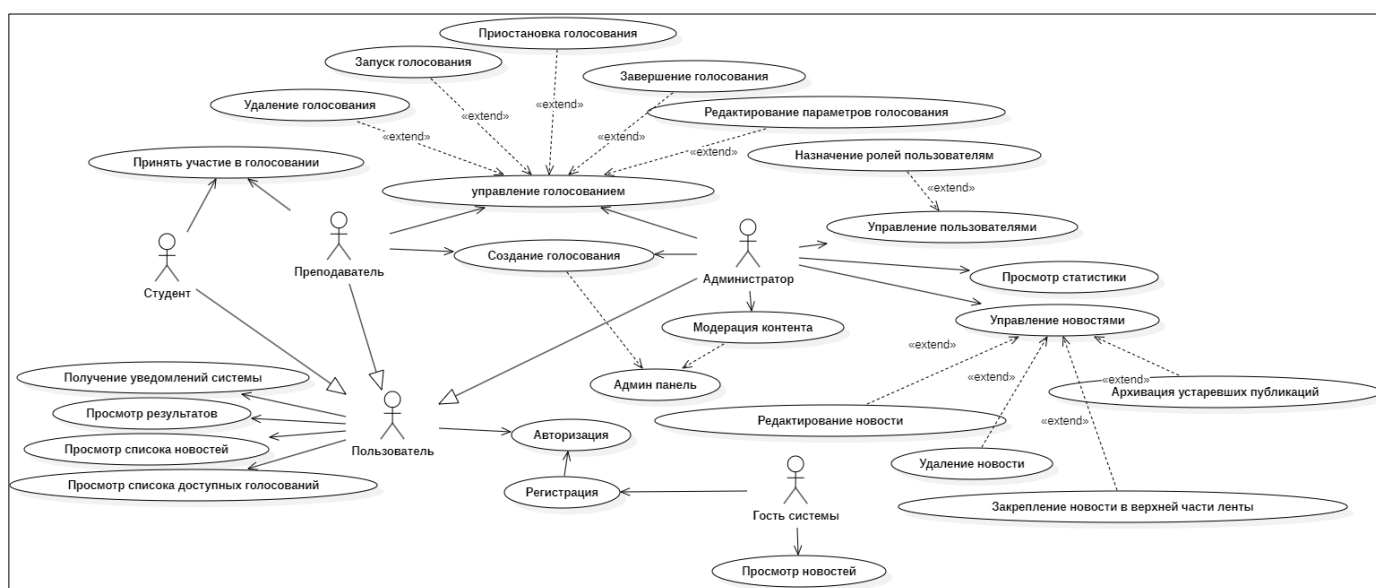


Рисунок 2.2 – Диаграмма прецедентов и актеров

В системе онлайн-голосования выделяется актёр «Гость», который представляет собой неавторизованного пользователя системы. Гость имеет доступ к ограниченному функционалу платформы, позволяющему ознакомиться с возможностями системы перед регистрацией. Он может просматривать новостную ленту системы, а также пройти процедуру регистрации для получения полного доступа к функционалу платформы.

Далее приведено описание прецедентов для роли «Гость».

Прецедент «Регистрация в системе» предназначен для создания новой учетной записи пользователя. В рамках данного прецедента пользователь вводит свои регистрационные данные, такие как адрес электронной почты, пароль и дополнительную информацию, необходимую для идентификации в системе. После заполнения формы регистрации система отправляет письмо подтверждения на указанный адрес электронной почты. После подтверждения регистрации пользователь получает доступ к функционалу системы.

Прецедент «Просмотр новостей» предназначен для ознакомления пользователя с новостями системы. Пользователь может просматривать список новостей, открывать полную версию публикации и просматривать прикрепленные материалы.

Пользователь является основным участником системы и представляет собой базовую роль, от которой наследуются другие роли системы, такие как студент, преподаватель и администратор. Пользователь может просматривать доступные голосования, просматривать результаты голосований, а также получать уведомления системы. При этом пользователю сохраняются все возможности, доступные ранее для роли «Гость», такие как просмотр новостной ленты и ознакомление с опубликованными материалами системы.

Функциональные возможности, предоставляемые роли «Пользователь», являются базовыми для всех авторизованных пользователей системы. Данные возможности также доступны пользователям с ролями «Студент», «Преподаватель» и «Администратор», которые, помимо базового функционала, обладают дополнительными правами и возможностями в соответствии со своими ролями.

Далее приведено описание прецедентов для роли «Пользователь».

Прецедент «Авторизация в системе» предназначен для предоставления пользователю доступа к функционалу системы на основании ранее созданной учетной записи. Пользователь вводит свой логин и пароль, после чего система проверяет корректность введенных данных и предоставляет доступ к личному кабинету. При необходимости может использоваться двухфакторная аутентификация для повышения уровня безопасности.

Прецедент «Просмотр списка доступных голосований» предназначен для ознакомления пользователя с текущими активными голосованиями. В рамках данного прецедента пользователь может просмотреть название голосования, его описание, сроки проведения и другие параметры.

Прецедент «Просмотр результатов голосования» предназначен для ознакомления пользователя с итогами проведенного голосования. Результаты могут отображаться в виде диаграмм, графиков или процентного распределения голосов.

Прецедент «Получение уведомлений» предназначен для информирования пользователя о важных событиях системы, таких как начало голосования, приближение

окончания голосования или публикация результатов. Уведомления могут отправляться через электронную почту или отображаться внутри системы.

Студент получает доступ к системе после регистрации и авторизации и обладает расширенными возможностями по сравнению с неавторизованным пользователем. При этом пользователю с ролью «Студент» доступны все функции, предоставляемые роли «Пользователь», включая просмотр новостей системы, получение уведомлений, а также доступ к общей информации платформы.

Дополнительно студент может участвовать в голосованиях, знакомиться с их описанием и параметрами.

Далее приведено описание прецедентов для роли «Студент».

Прецедент «Участие в голосовании» предназначен для предоставления пользователю возможности принять участие в голосовании. Пользователь выбирает один или несколько вариантов ответа в зависимости от типа голосования и подтверждает свой выбор. Система фиксирует голос пользователя и проверяет соблюдение ограничения «один пользователь – один голос».

Преподаватель отвечает за создание и управление голосованиями, а также может сам принимать участие в голосованиях. Также ему доступен весь функционал, который предоставляется базовой роли «Пользователь». А также преподаватель может создавать новые голосования, добавлять кандидатов или варианты выбора, настраивать параметры голосования (даты начала и окончания, тип голосования, анонимность).

Далее приведено описание прецедентов для роли «Преподаватель».

Прецедент «Создание голосования» предназначен для формирования нового голосования в системе. Организатор указывает название голосования, его описание, дату и время начала и окончания, тип голосования, а также список вариантов ответа. Дополнительно могут задаваться параметры анонимности голосования и ограничения по группе пользователей. При создании голосования оно проходит модерацию со стороны администратора.

Прецедент «Управление голосованием» предназначен для администрирования ранее созданных голосований и изменения их параметров в процессе работы системы. В рамках данного прецедента организатор может выполнять различные действия, направленные на настройку, запуск и завершение голосования, а также анализ его результатов.

Данный прецедент включает ряд расширений, которые позволяют выполнять отдельные операции управления голосованием.

Прецедент «Редактирование параметров голосования» предназначен для изменения ранее созданного голосования. Организатор может изменить описание, дату проведения, варианты ответа и другие параметры голосования.

Прецедент «Запуск голосования» предназначен для активации созданного голосования. После запуска голосования пользователи получают возможность принять в нём участие.

Прецедент «Приостановка голосования» предназначен для временного прекращения процесса голосования. Данный прецедент может использоваться при обнаружении ошибок или необходимости внести изменения в параметры голосования.

Прецедент «Завершение голосования» предназначен для окончательного завершения процесса голосования. После завершения система автоматически выполняет подсчёт голосов и формирует статистику результатов.

Прецедент «Удаление голосования» предназначен для удаления ранее созданного голосования из системы. Данный прецедент позволяет организатору удалять голосования, которые больше не являются актуальными или были созданы ошибочно.

Администратор выполняет функции управления системой. Он контролирует пользователей, управляет ролями, может публиковать новости в новостной ленте системы, а также отслеживает действия пользователей через систему аудита. Ему также доступен весь функционал с управлением голосованиями.

Далее приведено описание прецедентов для роли «Администратор»

Прецедент «Управление пользователями» предназначен для администрирования учетных записей пользователей системы. В рамках данного прецедента администратор может добавлять новых пользователей, редактировать их данные, блокировать или удалять учетные записи.

Прецедент «Назначение ролей пользователям» предназначен для управления правами доступа пользователей. Администратор может назначать пользователям различные роли, такие как преподаватель, организатор или участник, что определяет набор доступных им функций в системе. Данный прецедент является расширением прецедента «Управление пользователями», поскольку назначение ролей выполняется в рамках управления учетными записями пользователей.

Прецедент «Модерация контента» предназначен для контроля содержимого системы. Администратор может проверять созданные голосования, удалять некорректные данные и контролировать корректность публикуемого контента.

Прецедент «Просмотр общей статистики системы» предназначен для анализа активности пользователей и работы системы в целом. Администратор может просматривать количество пользователей, количество проведённых голосований и другие показатели.

Прецедент «Управление новостями» предназначен для администрирования новостных публикаций системы. В рамках данного прецедента администратор может создавать новые новости, редактировать существующие публикации, управлять их отображением в новостной ленте, а также удалять или архивировать устаревшие материалы. Данный прецедент включает ряд расширений, позволяющих выполнять отдельные операции управления новостным контентом.

Прецедент «Редактирование новости» предназначен для внесения изменений в уже опубликованные новости. Администратор может изменять заголовок новости, текст публикации, прикрепленные материалы и другие параметры.

Прецедент «Удаление новости» предназначен для удаления новостной публикации из системы в случае её неактуальности, ошибки при публикации или нарушения правил размещения контента.

Прецедент «Закрепление новости в верхней части ленты» предназначен для выделения наиболее важной публикации. Закреплённая новость отображается в верхней части новостной ленты независимо от даты её публикации.

Прецедент «Архивация устаревших публикаций» предназначен для перемещения неактуальных новостей в архив системы. Архивирование позволяет сохранить публикации для последующего просмотра, не отображая их в основной новостной ленте.

2.3 Информационная модель программного комплекса

Одной из ключевых задач большинства современных веб-приложений является хранение и обработка данных. Базы данных являются неотъемлемой частью функционирования подавляющего большинства программных систем, поскольку позволяют эффективно организовывать хранение информации, обеспечивать её целостность, структурированность и быстрый доступ к необходимым данным.

В разрабатываемой системе онлайн-голосования в образовательной среде для хранения данных будет использоваться реляционная система управления базами данных *PostgreSQL*. Использование реляционной базы данных позволяет организовать структурированное хранение информации, обеспечить поддержку транзакций, целостность данных и выполнение сложных запросов, необходимых для функционирования системы.

При этом в системе будет применяться комбинированный подход к хранению данных. Часть информации, связанная непосредственно с процессом голосования и требующая обеспечения неизменяемости и прозрачности, будет храниться в сети блокчейн. К таким данным относятся сведения о поданных голосах, хэши результатов голосования и информация о транзакциях. Остальная часть данных, необходимая для функционирования приложения, будет храниться вне блокчейна (*off-chain*) в базе данных *PostgreSQL*. К таким данным относятся сведения о пользователях системы, структура образовательного учреждения, параметры голосований, новостные публикации и другие служебные данные. Использование такого подхода позволяет объединить преимущества блокчейн-технологий, обеспечивающих прозрачность и неизменяемость данных, с высокой производительностью и гибкостью реляционной базы данных.

Для заполнения базы данных используются данные, создаваемые непосредственно в процессе работы системы, а также тестовые данные, применяемые на этапе разработки и проверки работоспособности приложения. К таким данным относятся сведения о пользователях системы, голосованиях, вариантах ответов, результатах голосования, а также организационной структуре образовательного учреждения.

На основе анализа предметной области можно выделить следующие основные сущности системы:

- факультеты (*Faculties*);
- специальности (*Specialities*);
- кафедры или подразделения (*Departments*);
- курсы обучения (*Courses*);
- учебные группы (*Groups*);
- пользователи (*Users*);
- роли пользователей (*Roles*);
- голосования (*Votings*);
- кандидаты или варианты ответа (*Candidates*);

- голоса пользователей (*Votes*);
- цели голосования (*VotingTarget*);
- новости системы (*News*);
- уведомления (*Notifications*);
- журнал действий пользователей (*Audit_log*);
- записи о взаимодействии с блокчейном (*Blockchain_records*).

Сущности «*Faculty*», «*Speciality*», «*Department*», «*Course*» и «*Group*» отражают организационную структуру образовательного учреждения. Они используются для определения принадлежности пользователей к определённому факультету, кафедре, специальности, курсу и учебной группе.

Сущность «*User*» содержит информацию о пользователях системы. В данной таблице хранятся данные о пользователе, такие как электронная почта, адрес криптокошелька, роль пользователя в системе, принадлежность к учебной группе, а также дата регистрации.

Сущность «*Role*» предназначена для хранения ролей пользователей системы: студент, преподаватель, администратор системы. Использование отдельной таблицы ролей позволяет гибко управлять правами доступа пользователей.

Сущность «*Votings*» содержит информацию о проводимых голосованиях. В таблице хранятся такие параметры голосования, как название, описание, тип голосования, признак анонимности, время начала и окончания голосования, а также текущий статус голосования.

Сущность «*Candidates*» содержит информацию о кандидатах или вариантах ответа, которые участвуют в голосовании. Каждый кандидат связан с конкретным голосованием и может представлять либо пользователя системы, либо произвольный вариант ответа.

Сущность «*Votes*» предназначена для хранения информации о поданных голосах пользователей. В таблице фиксируется идентификатор голосования, выбранный кандидат, пользователь, отдавший голос, а также хэш голоса и время его создания.

Сущность «*VotingTarget*» используется для определения целевой аудитории голосования. С её помощью можно ограничить участие в голосовании пользователями определённого факультета, кафедры, курса, группы или роли.

Сущность «*News*» предназначена для хранения новостных публикаций системы. В таблице содержится информация о заголовке новости, её содержимом, авторе публикации, дате публикации, а также дополнительных вложениях.

Сущность «*Notifications*» используется для хранения уведомлений пользователей системы. Уведомления могут информировать пользователей о начале голосования, публикации новых новостей или других важных событиях системы.

Сущность «*Audit_log*» представляет журнал действий пользователей и предназначена для фиксации различных событий в системе, таких как создание голосований, изменение параметров или другие действия пользователей.

Сущность «*Blockchain_records*» используется для хранения информации о взаимодействии системы с блокчейн-сетью. В данной таблице сохраняются данные о транзакциях, связанных с голосованиями, включая хэш транзакции, хэш голосов, номер блока и время создания записи.

Таким образом, разработанная информационная модель базы данных позволяет эффективно хранить данные системы онлайн-голосования, обеспечивать целостность информации и поддерживать все основные функции приложения.

Далее рассмотрим схему базы данных более подробно. На рисунке 2.3 представлена схема базы данных системы онлайн-голосования.

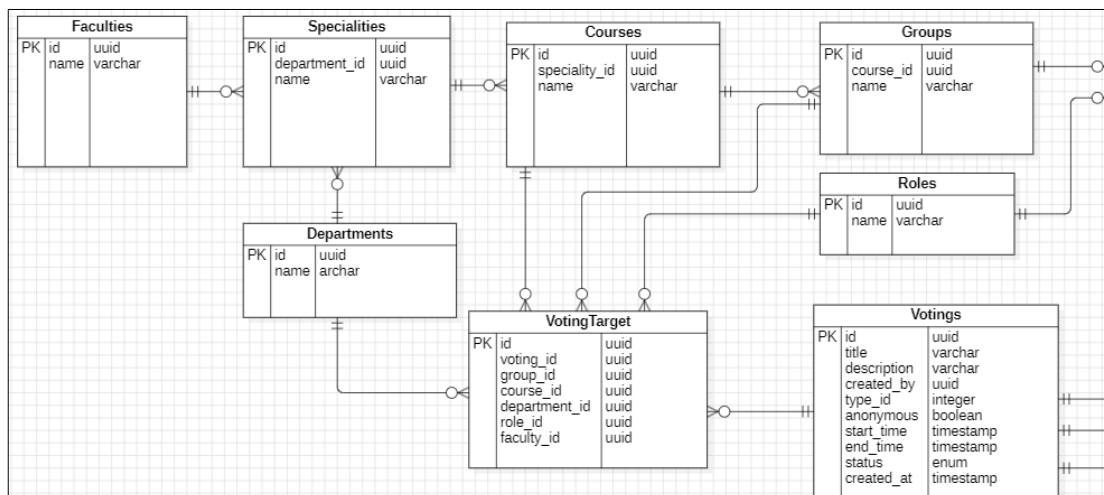


Рисунок 2.3 – Схема базы данных системы онлайн-голосования (часть 1)

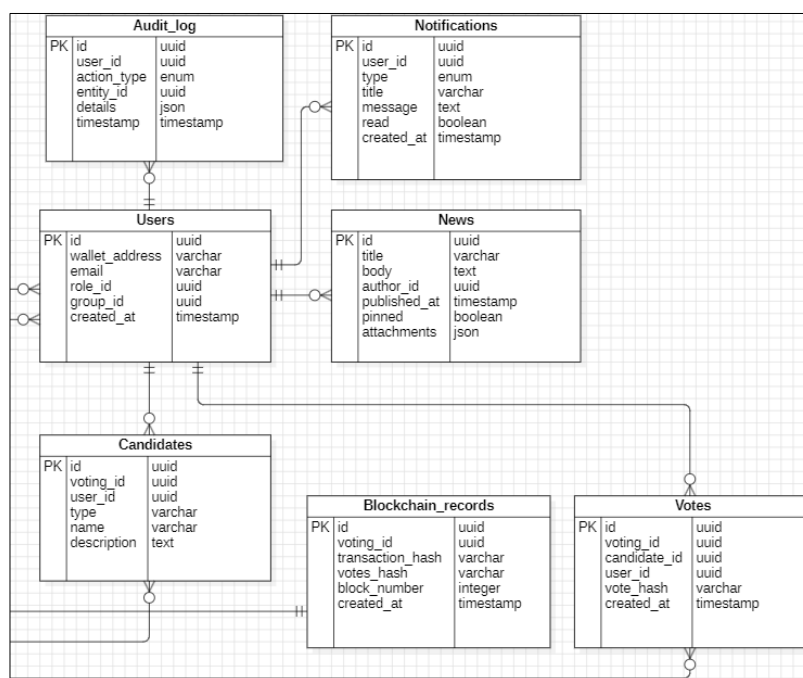


Рисунок 2.3 – Схема базы данных системы онлайн-голосования (часть 2)

На представленной схеме отображены основные таблицы базы данных, а также связи между ними. Каждая таблица содержит первичный ключ *id*, который представляет собой уникальный идентификатор записи и используется для однозначной идентификации данных и организации связей между таблицами.

В таблицах 2.1 – 2.15 представлено подробное описание полей в коллекциях.

Таблица 2.1 – Таблица «*Faculties*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор факультета	<i>uuid</i>	да	да
<i>name</i>	Название факультета	<i>varchar(255)</i>	да	да

Как видно из таблицы 2.1, таблица «*Faculties*» хранит информацию о факультетах образовательного учреждения. В данной таблице содержатся основные данные о факультетах, включая их уникальный идентификатор и название.

Для более детального описания структуры образовательного учреждения используется таблица «*Specialities*», которая содержит информацию о специальностях, реализуемых в рамках соответствующих подразделений.

Таблица 2.2 – Таблица «*Specialities*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор специальности	<i>uuid</i>	да	да
<i>department_id</i>	Ссылка на кафедру или подразделение	<i>uuid</i>	да	нет
<i>name</i>	Название специальности	<i>varchar(255)</i>	да	нет

Как видно из таблицы 2.2, таблица «*Specialities*» хранит информацию о специальностях образовательного учреждения. В данной таблице содержатся сведения о названии специальности, а также ссылка на подразделение, в рамках которого она реализуется.

Для описания кафедры образовательного учреждения, в рамках которых реализуются различные специальности, используется таблица «*Departments*».

Таблица 2.3 – Таблица «*Departments*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор кафедры	<i>uuid</i>	да	да
<i>name</i>	Название кафедры	<i>varchar(255)</i>	да	да

Как видно из таблицы 2.3, таблица «*Departments*» хранит информацию о кафедрах образовательного учреждения. В данной таблице содержатся данные о названии подразделения и его уникальном идентификаторе.

Для более точного определения этапа обучения студентов используется таблица «*Courses*», которая содержит информацию о курсах обучения.

Таблица 2.4 – Таблица «*Courses*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор курса обучения	<i>uuid</i>	да	да
<i>speciality_id</i>	Ссылка на специальность	<i>uuid</i>	да	нет
<i>name</i>	Номер или название курса	<i>varchar(255)</i>	да	нет

Как видно из таблицы 2.4, таблица «*Courses*» содержит информацию о курсах обучения. В таблице хранится идентификатор курса, его название, а также ссылка на соответствующую специальность.

Для еще более точной идентификации студентов в системе используется таблица «*Groups*», которая содержит информацию об учебных группах.

Таблица 2.5 – Таблица «*Groups*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор учебной группы	<i>uuid</i>	да	да
<i>course_id</i>	Ссылка на курс обучения	<i>uuid</i>	да	нет
<i>name</i>	Название или номер учебной группы	<i>varchar(255)</i>	да	нет

Как видно из таблицы 2.5, таблица «*Groups*» хранит информацию об учебных группах. В данной таблице содержатся данные о названии группы и курсе обучения, к которому она относится.

Для хранения информации о пользователях системы используется таблица «*Users*», которая содержит основные сведения о зарегистрированных пользователях.

Таблица 2.6 – Таблица «*Users*»

Название	Описание	Тип	Обязательное	Уникальное
1	2	3	4	5
<i>id</i>	Идентификатор пользователя	<i>uuid</i>	да	да

Продолжение таблицы 2.6

1	2	3	4	5
<i>created at</i>	Дата регистрации	<i>timestamp</i>	да	нет
<i>wallet_address</i>	Адрес кошелька пользователя	<i>varchar(255)</i>	да	да
<i>email</i>	<i>Email</i> пользователя	<i>varchar(255)</i>	нет	да
<i>role_id</i>	Роль пользователя	<i>integer</i>	да	нет
<i>group_id</i>	Ссылка на группу пользователя	<i>uuid</i>	нет	нет

Как видно из таблицы 2.6, таблица «*Users*» хранит информацию о пользователях системы. В данной таблице содержатся основные сведения о пользователях, включая уникальный идентификатор, адрес криптовалютного кошелька, адрес электронной почты, роль пользователя в системе, а также принадлежность к учебной группе.

Для управления правами доступа пользователей в системе используется таблица «*Roles*», которая содержит перечень ролей пользователей.

Таблица 2.7 – Таблица «*Roles*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор роли	<i>integer</i>	да	да
<i>name</i>	Название роли (<i>user, teacher, admin</i>)	<i>varchar(50)</i>	да	да

Как видно из таблицы 2.7, таблица «*Roles*» хранит информацию о ролях пользователей системы. В данной таблице содержатся данные о названиях ролей, которые определяют уровень доступа пользователей к функциональным возможностям системы.

Для хранения информации о проводимых голосованиях используется таблица «*Votings*».

Таблица 2.8 – Таблица «*Votings*»

Название	Описание	Тип	Обязательное	Уникальное
1	2	3	4	5
<i>id</i>	Идентификатор голосования	<i>uuid</i>	да	да
<i>title</i>	Название голосования	<i>varchar(255)</i>	да	нет
<i>description</i>	Описание голосования	<i>text</i>	нет	нет
<i>created_by</i>	Организатор голосования	<i>uuid</i>	да	нет
<i>type_id</i>	Тип голосования	<i>integer</i>	да	нет
<i>anonymous</i>	Анонимное голосование	<i>boolean</i>	да	нет

Продолжение таблицы 2.8

1	2	3	4	5
<i>start time</i>	Время начала	<i>timestamp</i>	да	нет
<i>end time</i>	Время окончания	<i>timestamp</i>	да	нет
<i>status</i>	Статус (<i>draft, active, finished</i>)	<i>varchar(50)</i>	да	нет
<i>created at</i>	Дата создания	<i>timestamp</i>	да	нет

Как видно из таблицы 2.8, таблица «*Votings*» хранит информацию о голосованиях, проводимых в системе. В данной таблице содержатся такие параметры, как название голосования, его описание, автор создания, признак анонимности, время начала и окончания голосования, а также текущий статус голосования.

Для хранения информации о кандидатах или вариантах ответов, участвующих в голосовании, используется таблица «*Candidates*».

Таблица 2.9 – Таблица «*Candidates*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор кандидата	<i>uuid</i>	да	да
<i>voting id</i>	Голосование	<i>uuid</i>	да	нет
<i>user_id</i>	Ссылка на пользователя, если кандидат человек	<i>uuid</i>	нет	нет
<i>type</i>	Тип кандидата (<i>user, project, report</i>)	<i>varchar(50)</i>	да	нет
<i>name</i>	Имя кандидата / название объекта	<i>varchar(255)</i>	да	нет
<i>description</i>	Описание кандидата / проекта	<i>text</i>	нет	нет

Как видно из таблицы 2.9, таблица «*Candidates*» хранит информацию о кандидатах или вариантах ответов, которые участвуют в голосовании. В данной таблице содержатся данные о пользователе-кандидате, типе кандидата, его названии и описании.

Для фиксации результатов голосования пользователей используется таблица «*Votes*».

Таблица 2.10 – Таблица «*Votes*»

Название	Описание	Тип	Обязательное	Уникальное
1	2	3	4	5
<i>id</i>	Идентификатор голоса	<i>uuid</i>	да	да
<i>voting id</i>	Голосование	<i>uuid</i>	да	нет
<i>candidate id</i>	Выбранный кандидат	<i>uuid</i>	да	нет

Продолжение таблицы 2.10

1	2	3	4	5
<i>user_id</i>	Пользователь, который голосует	<i>uuid</i>	да	нет
<i>vote_hash</i>	Хэш голоса	<i>varchar(255)</i>	да	да
<i>created_at</i>	Время голосования	<i>timestamp</i>	да	нет

Как видно из таблицы 2.10, таблица «*Votes*» хранит информацию о поданных голосах пользователей. В данной таблице фиксируются сведения о голосовании, выбранном кандидате, пользователе, который отдал голос, а также хэш голоса.

Для определения целевой аудитории голосований используется таблица «*VotingTarget*».

Таблица 2.11 – Таблица «*VotingTarget*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор записи ограничения голосования	<i>uuid</i>	да	да
<i>voting_id</i>	Ссылка на голосование	<i>uuid</i>	да	нет
<i>group_id</i>	Ссылка на учебную группу, для которой доступно голосование	<i>uuid</i>	нет	нет
<i>course_id</i>	Ссылка на курс обучения, для которого доступно голосование	<i>uuid</i>	нет	нет
<i>department_id</i>	Ссылка на кафедру или подразделение	<i>uuid</i>	нет	нет
<i>role_id</i>	Ссылка на роль пользователя	<i>uuid</i>	нет	нет
<i>faculty_id</i>	Ссылка на факультет	<i>uuid</i>	нет	нет

Для хранения новостных публикаций системы используется таблица «*News*».

Таблица 2.12 – Таблица «*News*»

Название	Описание	Тип	Обязательное	Уникальное
1	2	3	4	5
<i>id</i>	Идентификатор новости	<i>uuid</i>	да	да
<i>title</i>	Заголовок	<i>varchar(255)</i>	да	нет

Продолжение таблицы 2.12

1	2	3	4	5
<i>body</i>	Текст новости	<i>text</i>	да	нет
<i>author id</i>	Автор	<i>uuid</i>	да	нет
<i>published_at</i>	Дата публикации	<i>timestamp</i>	да	нет
<i>pinned</i>	Закрепить новость	<i>boolean</i>	да	нет
<i>attachments</i>	Файлы / вложения	<i>json</i>	нет	нет

Как видно из таблицы 2.12, таблица «*News*» хранит информацию о новостных публикациях системы. В данной таблице содержатся данные о заголовке новости, её содержимом, авторе публикации, дате публикации, а также дополнительных вложениях.

Для уведомления пользователей о различных событиях системы используется таблица «*Notifications*».

Таблица 2.13 – Таблица «*Notifications*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор уведомления	<i>uuid</i>	да	да
<i>user id</i>	Пользователь	<i>uuid</i>	да	нет
<i>type</i>	Тип уведомления (<i>email, system</i>)	<i>varchar(50)</i>	да	нет
<i>title</i>	Заголовок уведомления	<i>varchar(255)</i>	да	нет
<i>message</i>	Текст уведомления	<i>text</i>	да	нет
<i>read</i>	Прочитано	<i>boolean</i>	да	нет
<i>created_at</i>	Время создания	<i>timestamp</i>	да	нет

Как видно из таблицы 2.13, таблица «*Notifications*» хранит информацию об уведомлениях пользователей системы. В данной таблице содержатся данные о типе уведомления, его заголовке, тексте сообщения, а также статусе прочтения.

Для фиксации действий пользователей в системе используется таблица «*Audit_log*».

Таблица 2.14 – Таблица «*Audit_log*»

Название	Описание	Тип	Обязательное	Уникальное
1	2	3	4	5
<i>id</i>	Идентификатор записи	<i>uuid</i>	да	да
<i>user id</i>	Пользователь	<i>uuid</i>	да	нет

Продолжение таблицы 2.14

1	2	3	4	5
<i>action_type</i>	Тип действия (<i>login</i> , <i>vote</i> , <i>create_voting</i> , <i>edit_voting</i> и т.д.)	<i>varchar(100)</i>	да	нет
<i>entity_id</i>	Ссылка на объект действия (<i>voting</i> , <i>candidate</i> , <i>news</i>)	<i>uuid</i>	нет	нет
<i>details</i>	Дополнительная информация (<i>json</i>)	<i>json</i>	нет	нет
<i>timestamp</i>	Время действия	<i>timestamp</i>	да	нет

Как видно из таблицы 2.14, таблица «*Audit_log*» хранит информацию о действиях пользователей в системе. В данной таблице фиксируются данные о пользователе, типе выполненного действия, объекте, над которым было выполнено действие, а также дополнительные сведения и время выполнения операции.

Для хранения информации о взаимодействии системы с блокчейн-сетью используется таблица «*Blockchain_records*».

Таблица 2.15 – Таблица «*Blockchain_records*»

Название	Описание	Тип	Обязательное	Уникальное
<i>id</i>	Идентификатор записи	<i>uuid</i>	да	да
<i>voting_id</i>	Голосование	<i>uuid</i>	да	да
<i>transaction_hash</i>	Хэш транзакции в блокчейне	<i>varchar(255)</i>	да	да
<i>votes_hash</i>	Хэш всех голосов	<i>varchar(255)</i>	да	нет
<i>block_number</i>	Номер блока	<i>integer</i>	нет	нет
<i>created_at</i>	Время записи в блокчейн	<i>timestamp</i>	да	нет

Как видно из таблицы 2.15, таблица «*Blockchain_records*» хранит информацию о взаимодействии системы с блокчейн-сетью. В данной таблице содержатся сведения о транзакциях, связанных с голосованиями, включая хэш транзакции, хэш голосов, номер блока и время создания записи.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была разработана система онлайн-голосования, обеспечивающая проведение опросов и голосований с возможностью последующего анализа результатов. Основной целью разработки являлось создание надёжного, удобного и масштабируемого программного решения, позволяющего организовывать голосования с обеспечением прозрачности и неизменяемости итоговых данных.

В процессе работы были проанализированы существующие подходы к реализации систем голосования, а также определены основные требования к разрабатываемому приложению. На основе проведённого анализа была сформирована архитектура системы, включающая клиентскую часть, серверную часть, базу данных и интеграцию с блокчейн-сетью *Ethereum* (тестовая сеть *Sepolia*).

В рамках проектирования была разработана логическая модель предметной области, определены основные роли пользователей системы, а также сформированы сценарии взаимодействия пользователей с приложением. Для визуализации функциональной структуры использовались *UML*-диаграммы, построенные с помощью *StarUML*.

В результате разработки была реализована система, позволяющая пользователям принимать участие в голосованиях, организаторам – создавать и настраивать голосования, а администраторам – управлять системой и контролировать её работу. Основные данные голосований хранятся в базе данных, что обеспечивает высокую производительность и удобство обработки информации.

Для повышения надёжности и обеспечения неизменяемости результатов голосования была реализована интеграция с блокчейном. В системе используется механизм сохранения хэш-суммы результатов голосования в сети блокчейн, что позволяет гарантировать целостность данных и возможность их независимой проверки.

Разработанное решение отличается рациональным использованием технологий: применение блокчейна ограничено фиксацией итоговых данных, что позволяет минимизировать затраты на выполнение транзакций и повысить общую эффективность системы.

Таким образом, поставленная цель дипломного проекта была достигнута. Разработанная система онлайн-голосования соответствует предъявляемым требованиям, обладает необходимым функционалом и может быть использована в качестве основы для дальнейшего развития и внедрения в практическую деятельность.

Список использованных источников

1. Каюмов В.В. *JavaScript*: программирование. – М.: БХВ-Петербург, 2019. – 512 с.
2. Васильев А.С. *React.js*: разработка интерфейсов. – СПб.: Питер, 2020. – 384 с.
3. Троелсен Э., Хорспул Дж. *C# 10 и .NET 6*. Современная кроссплатформенная разработка. – М.: ДМК Пресс, 2022. – 1424 с.
4. Олсон Р. *PostgreSQL 14*. Администрирование и разработка. – М.: Эксмо, 2022. – 576 с.
5. Монтгомери Д. *Pro Entity Framework Core 6*. – М.: ДМК Пресс, 2022. – 432 с.
6. Мазер Э., Вул Х., Шнайер Б. Криптография для разработчиков. – М.: Танграм Паблик, 2021. – 368 с.
7. Вуд Г. *Ethereum*: блокчейн-приложение и смарт-контракты. – М.: Альпина Паблишер, 2020. – 256 с.
8. Козлов М.А. Интегрированные среды разработки на платформе *.NET*. – М.: Горячая линия – Телеком, 2021. – 288 с.
9. Канер Д., Мерсье Х. *Docker*. Полное руководство. – СПб.: Питер, 2020. – 432 с.
10. Леминг Д., Элларт К. Версионный контроль с *Git*. – М.: Эксмо, 2019. – 496 с.
11. Грэнвилл Дж. *Postman*: тестирование API. – М.: БХВ-Петербург, 2021. – 192 с.
12. Каюмова А.В. Визуальное моделирование систем в *StarUML*: учебное пособие. – Казань: Казанский федеральный университет, 2013. – 104 с.